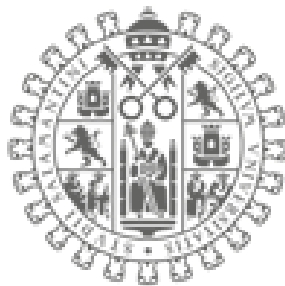


Calidad y Auditoría

Detección de errores, defectos y fallos de software



**VNiVERSiDAD
D SALAMANCA**

CAMPUS OF INTERNATIONAL EXCELLENCE

Miguel Barranquero Diez

Índice

1. Introducción	3
2. Programa 1	3
3. Programa 2	4
4. Programa 3	5
5. Programa 4	6
6. Programa 5	7
Bibliografía	8

1. Introducción

El presente trabajo se centra en el análisis y la corrección de defectos en cinco programas software proporcionados. La estructura de este ejercicio se basa en la aplicación de una metodología de prueba y depuración específica para cada código fuente. Esta metodología implica:

- Identificación estática de defectos.
- Definición de casos de prueba clave para exponer fallos.
- Validar el comportamiento esperado y aislar estados de error
- Corrección de los defectos identificados para asegurar la funcionalidad y verificar que el código modificado produce el resultado esperado.

2. Programa 1

La Figure 1 corresponde con el primer programa a analizar:

```
public class Array1 {

    public static void main(String[] args) {

        //Devuelve el número de ocurrencias de 0 en x

        int x[]={1,0,0};
        int count=0;
        for (int i=1 ; i<x.length; i++)
        {
            if (x[i] == 0)
            {
                count ++;
            }
        }
        System.out.print ("Número de ceros: ");
        System.out.println(count);
    }

}
```

Figure 1: Programa 1

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

3. Programa 2

La Figure 2 corresponde con el primer programa a analizar:

```
public class Array2 {
    public int findLast (int[] x, int y) {
        //Devuelve el índice del último elemento en x que sea igual a
        //Si no existe dicho elemento devuelve -1

        for (int i=x.length-1;i>0; i--)
        {
            if (x[i] == y)
            {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int x[]={0,1,1};
        int y=1;

        Array2 a=new Array2();
        int resultado = a.findLast (x,y);
        System.out.print ("El índice del elemento buscado es: ");
        System.out.print (resultado);
    }
}
```

Figure 2: Programa 2

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

4. Programa 3

La Figure 3 corresponde con el primer programa a analizar:

```
public class Array3 {  
    public int lastZero (int[] x) {  
        //Devuelve el índice del último 0 en x  
        //Si no existe 0 devuelve -1  
  
        for (int i=0; i<x.length-1; i++)  
        {  
            if (x[i] == 0)  
            {  
                return i;  
            }  
        }  
        return -1;  
    }  
  
    public static void main(String[] args) {  
  
        int x[]={0,1,0};  
  
        Array3 a=new Array3();  
        int resultado = a.lastZero (x);  
        System.out.print ("El índice del último 0 es: ");  
        System.out.print (resultado);  
    }  
}
```

Figure 3: Programa 3

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

5. Programa 4

La Figure 4 corresponde con el primer programa a analizar:

```
public class Array4 {  
    public int countPositive (int[] x) {  
        //Devuelve el número de elementos positivos en x  
        int count=0;  
        for (int i=0; i<x.length; i++)  
        {  
            if (x[i] >= 0)  
            {  
                count++;  
            }  
        }  
        return count;  
    }  
}  
  
    public static void main(String[] args) {  
  
        int x[]={-4,2,0,2};  
  
        Array4 a=new Array4();  
        int resultado = a.countPositive (x);  
        System.out.print ("El número de elementos positivos es: ");  
        System.out.print (resultado);  
    }  
}
```

Figure 4: Programa 4

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

6. Programa 5

La Figure 5 corresponde con el primer programa a analizar:

```
public class Array5 {  
    public int oddOrPositive (int[] x) {  
        //Devuelve el número de elementos en x que son impares o positivos  
        int count=0;  
        for (int i=0; i<x.length; i++)  
        {  
            if (x[i]%2 == 1 || x[i]>0)  
            {  
                count++;  
            }  
        }  
        return count;  
    }  
  
    public static void main(String[] args) {  
  
        int x[]={-4,2,1,2};  
  
        Array5 a=new Array5();  
        int resultado = a.oddOrPositive (x);  
        System.out.print ("El número de elementos impares o positivos es: ");  
        System.out.print (resultado);  
    }  
}
```

Figure 5: Programa 5

Identificar defectos (sin ejecutar el código)

Identificar un caso de prueba que dé lugar a un fallo

Si es posible, identificar el caso de prueba que no dé lugar a un fallo

Si es posible, identificar el caso de prueba no produzca un estado de error

Si es posible, identificar el caso de prueba que produzca un estado de error pero no produzca un fallo

Corregir el defecto y verificar que el caso de prueba identificado en el punto 2 produce el resultado esperado

Bibliografía